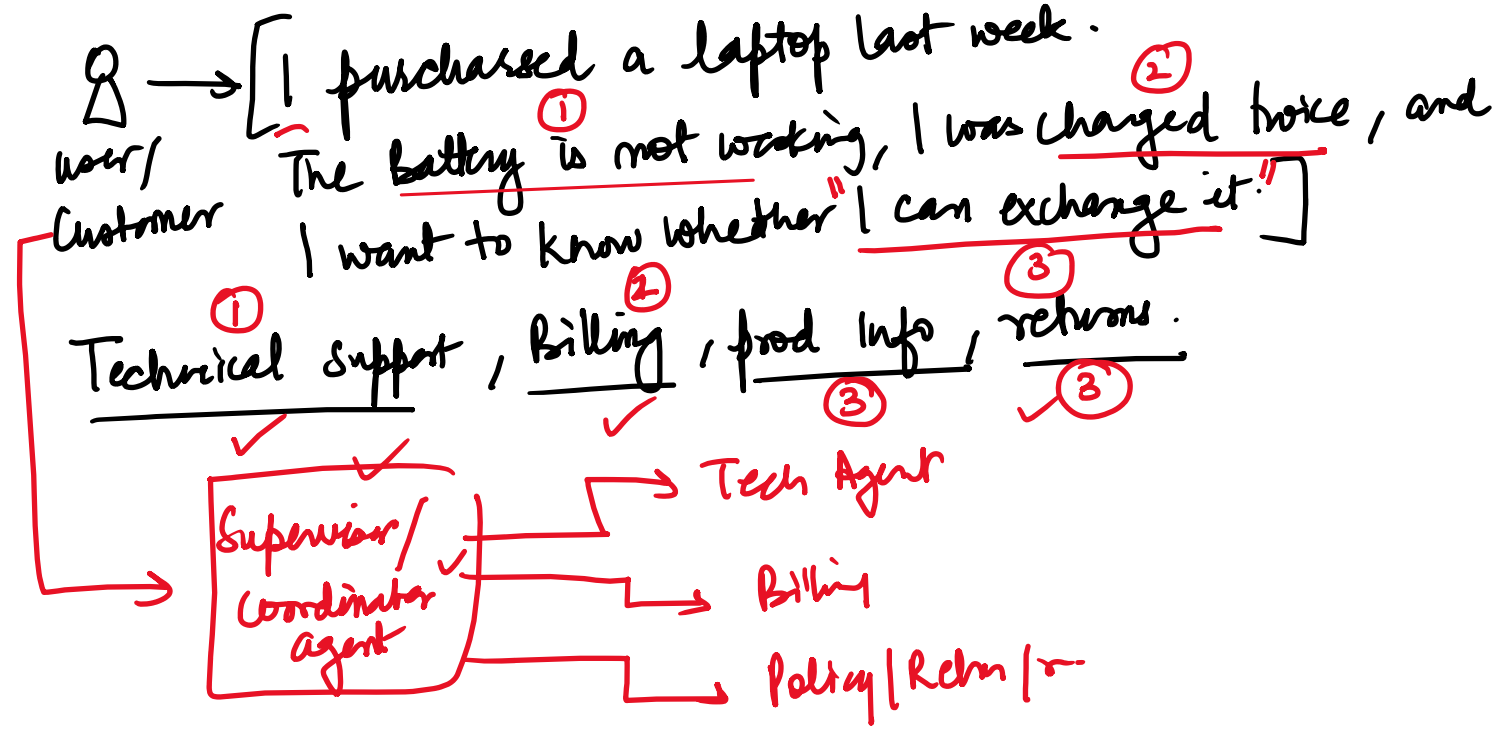
AWS AI/ML Stack

- Amazon Bedrock ⇒ 270+ FMs/LLMs → Development
- Amazon Bedrock AgentCore ? Deployment + Scale
- open-source frameworks - Langchain, STRANDS



Amazon Bedrock AgentCore - Runtime

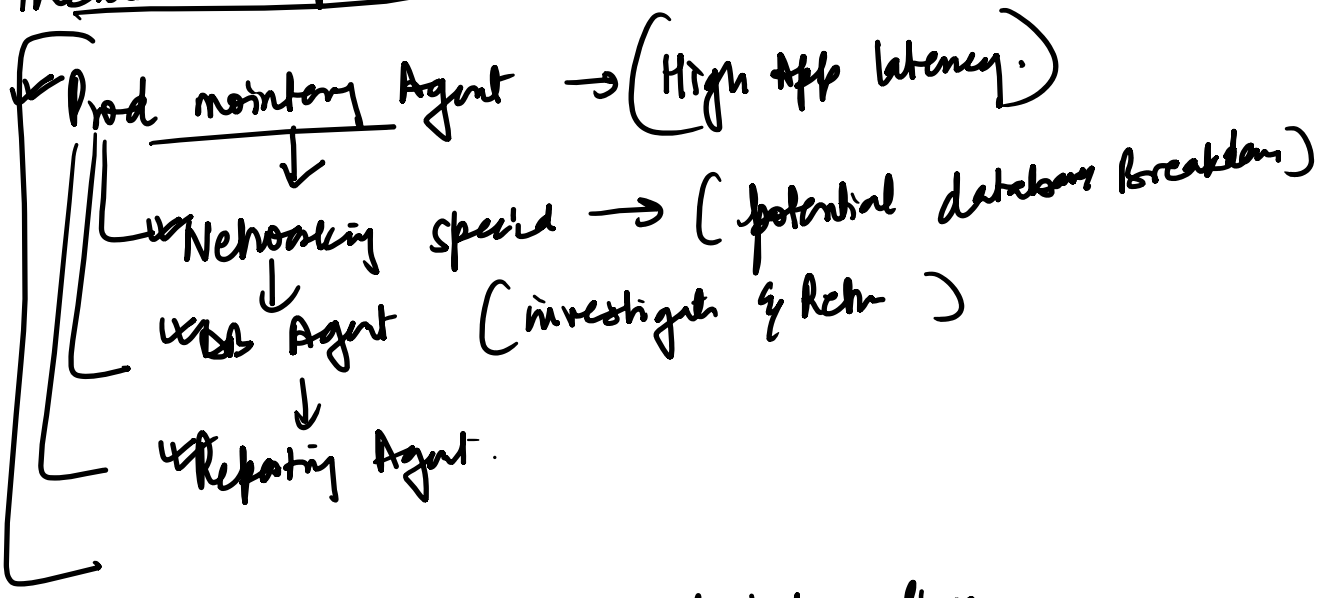
① Managed Hosting & Scaling

Calibration/Validation layer

- Evidence
- Consistency checks
- Confidence

Incident Response

Incident Response



Context Info = The current interaction

Persistent memory = Info that remains available for later interactions.

AgentCore Memory

1) Memory Resource = Managed Container
2) Actor = the entity to which events belong.
3) Session group = a particular interaction
4) Namespace = long-term memory records

These identifies determine How CONTEXT is isolated & retrieved.

Create_memory_session = a session for memory operate.

actor-id = memory events

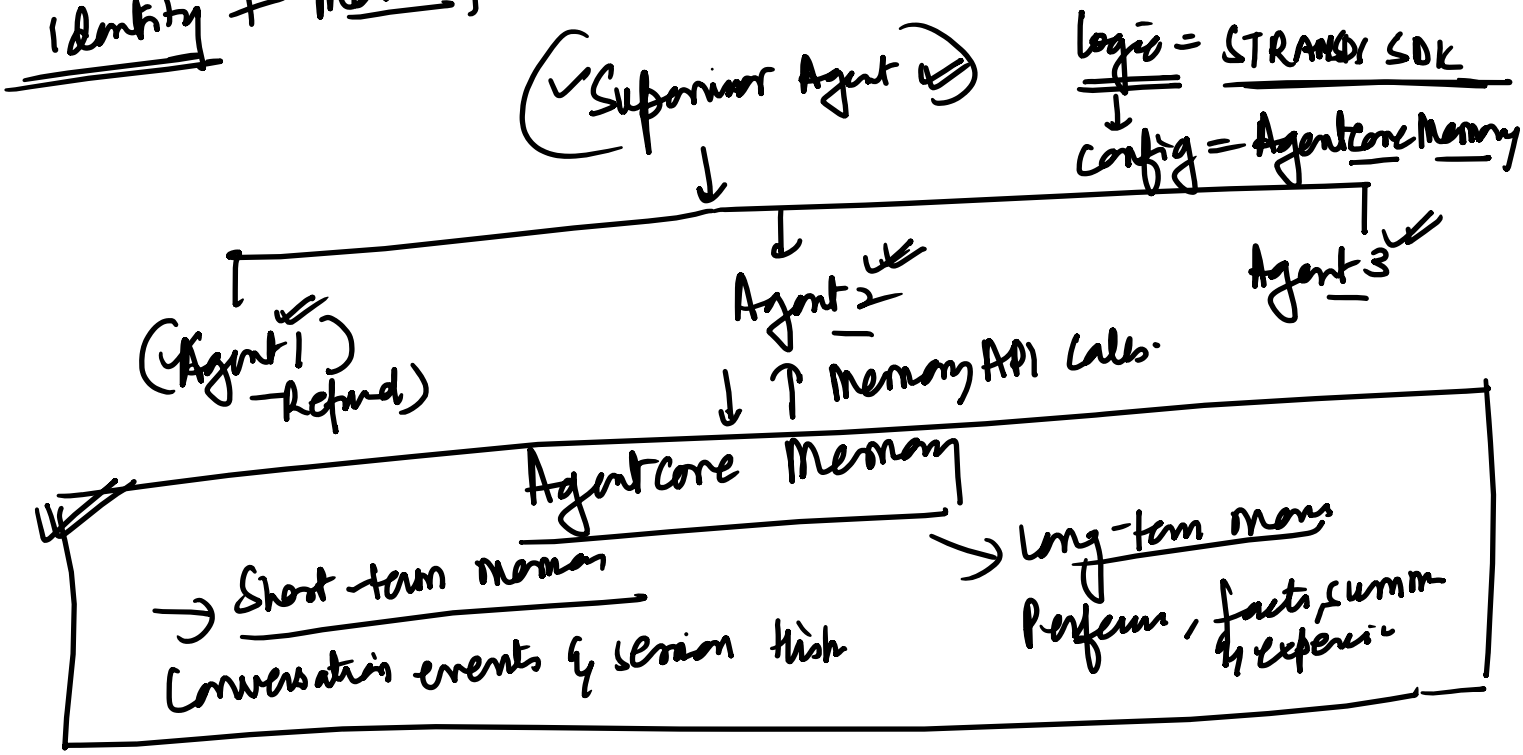
session-id = Interaction session

Short_term_Hooks = Connects agent lifecycle operate to Short-term memory

Short-term Hooks = Context ... memory

Agent = Sys Prompt + agent (Qwen)

Identity + Memory



Shared Memory / Isolated memory

Design	Configuration	Behavior
Shared session memory	→ Same memory resource, "short-term" actorId & sessionId	→ Agents can retrieve the same session events if authorized (identity)
Agent-specific memory	→ actorId & sessionId → Separate actor/session combinations	→ each agent maintains independent working context
Shared Long-term memory	→ same accessible "namespace"	→ Agents can reuse stored insights
Private Long-term memory	→ Separate "namespaces" with IAM restrictions	→ Limit access to authorized agents

... "memory configuration"

memory

Example "memory configuration"

Shared workflow context:

memoryresource → FinAdvisorMemory
ActorID — custom123
SessionID — workflow-456

Long-term namespaces

Shared facts : /customer/customer-123/shared/
Research agent's knowledge = /customer/customer-123/research

Q* App what to capture ? → Raw conv, Tool results, data, agent-generated summ
* memory (AgentCore)
Q. what to remember → Imp facts, preferences, decisions, exp, extracted through memory
Q* App what to share ? Which parts of that memory another agent needs for its current task.
↳ Imp decisions.

AgentCore provides Built-in memory

Strategy	What gets remembered	Example
✓ Semantic	- Facts & useful knowledge	- Customer has a credit limit of 5K
✓ Summarization	- Imp conversation topics, activities, decisions	- Customer requested for a loan eligibility Assessment
✓ user preference	- user preferences, specific choices	→ email communication.

✓ Episodic
- User preferences / choices
- Significant experiences & outcomes
- Semantic

Practical example: What should agents actually share?

3 agents procesing a loan application

Q → "I want to apply for a $10K loan. My salary is $5K. I have an existing car loan with an EMI $1.5K. I prefer 5 year repayment period.

Customer Service Agent → Strands SDK, Agentcore SDK
(Receive the conversation & record relevant facts)

Agentcore Memory (Blackboard)

Context
- Loan Amount =
- Income =
- Existing EMI =
- Prefer tenure =

Risk Agent
Income, EMI, loan Amount relevent verified Risk data

Recommendation Agent
Preferred tenure, loan Amt, Approved affordable amount

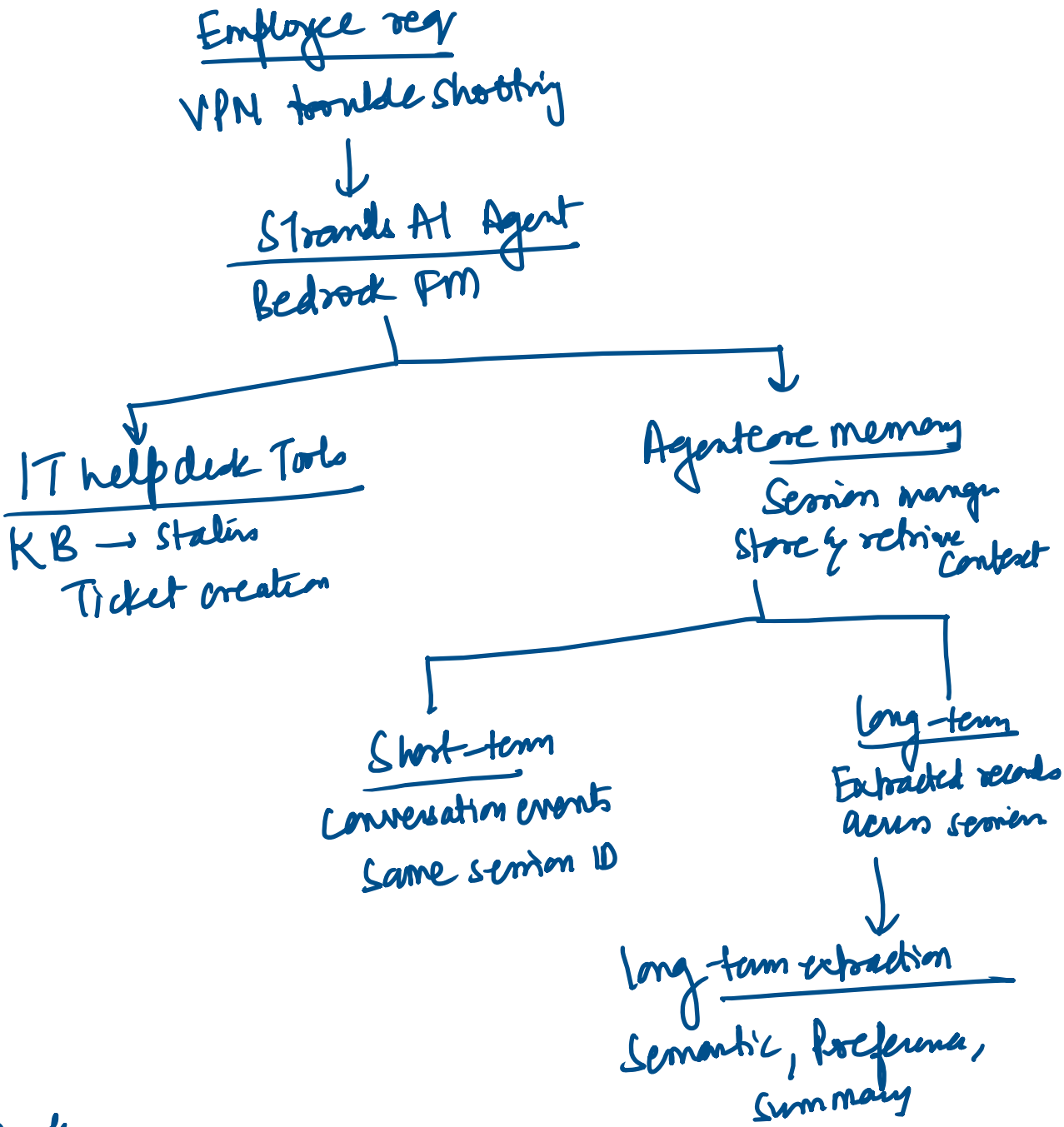
Demo: Context-aware Agent with Bedrock Agentcore memory.

(1) Create a Agentcore Memory resource configured with Semantic, User preference, Summary strategies.

(2) Integrate persistent memory into STRANDS agent & long-term retrieval to enable cross-

(2) Integrate persistent memory configure long-term retrieval to enable cross-session recall for a returning user.

(3) Inspect stored memories in the AgentCore Memory console.



NoteBook

(1) Baseline IT Helpdesk Agent → Agent, model, sys prompt, tools

(2) Agentcore Memory resource → Semantic, preference, summary

(2) AgentCore Memory resource → Semantic, preference, summary

(3) Memory enabled strands Agents
↳ Session Manager, actor ID, session ID, short term memory

(4) Two sessions with the same employee = Long-term extraction

Monday:

Secure AI Agent Access with AgentCore Identity

(1) Deploy an agent to Amazon Bedrock AgentCore Runtime with AgentCore Identity inbound Auth.

(2) Amazon Cognito user pool, get a JWT, decode it to see the claims AgentCore Identity checks

(3) Confirm AgentCore Identity reject callers without a valid JWT & lets valid ones through.

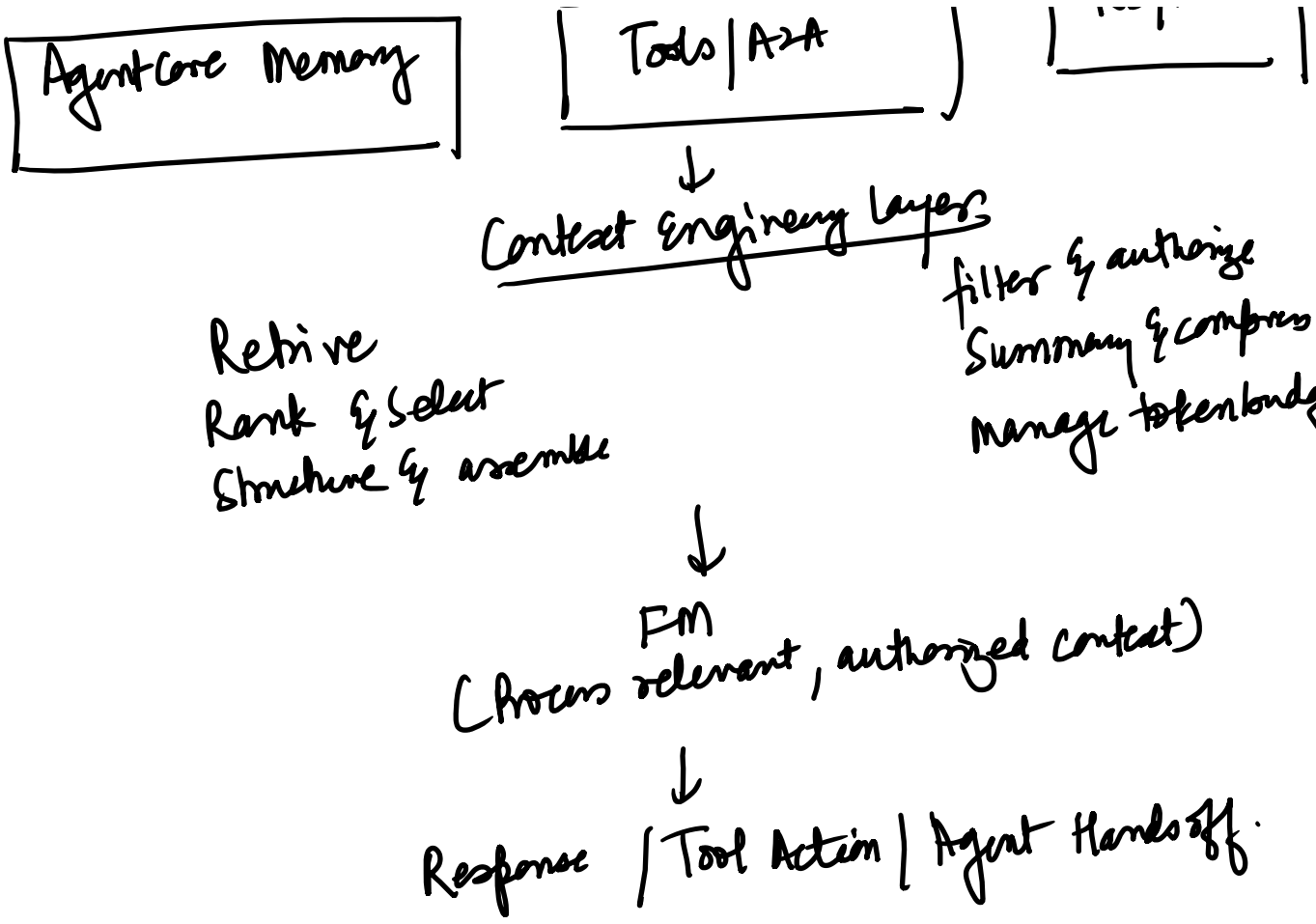
Context engineering in Generative AI & Multi-Agent Systems

Process of designing, selecting, organizing, managing the info to the AI Model / Agent so that it can produce accurate, relevant & reliable results.

What info should each agent receive, at what time, in what format, for what purpose?

Agent / Current Task

AgentCore Memory	Tools / A2A	KB / RAG


Agentcore Memory
Tools | A2A
Context Engineering Layers
Retrive
Rank & Select
Structure & assemble
filter & authorize
Summary & compress
manage tokenbudget
FM
(Process relevant, authorized context)
Response | Tool Action | Agent Handoff.